



Universidad Tecnológica de Bolívar

UNIVERSIDAD TECNOLÓGICA DE BOLÍVAR

Taller DYT: VRP BOARD

Presentado por:

German Eduardo De Armas Castaño
T00068765

Profesor:

Jaime Antonio Acevedo Chedid

March 2, 2026

Contents

1	Introducción	2
2	Enunciado y recursos	2
3	Generación de datos	2
4	Diseño de la ruta con un vehículo	2
5	Diseño con flota de tres vehículos	3
6	Demanda en Excel y cálculo de vehículos necesarios	3
7	Mapeo a ciudad real y ajuste de rutas	4
7.1	Primer Código: Extracción de Coordenadas (OpenCV)	4
7.2	Segundo Código: Limpieza y Agrupamiento (Pandas & Scikit-learn)	4
8	Selección y análisis de rutas	7
8.1	Preparación del Entorno Geoespacial	7
8.2	Normalización y Mapeo (Píxeles a Coordenadas GPS)	7
8.3	Resolución de Rutas y Métricas Logísticas	7
8.4	Visualización Cartográfica	7
9	Entregables	8
9.1	Selección e Identificación de Recorridos	9

1 Introducción

Este informe recoge la actividad práctica correspondiente al Taller de Distribución y Transporte (DYT). Se emplea la plataforma VRP indicada por el enunciado para generar una nube de clientes y diseñar rutas de reparto desde un depósito. El objetivo es implementar y documentar las siguientes tareas: generación de datos, diseño de rutas para un vehículo y para una flota de tres vehículos, estimación de vehículos necesarios según demanda y el mapeo de la solución sobre una ciudad real.

2 Enunciado y recursos

- Plataforma: VRP board.
- Caso base: 45 clientes generados aleatoriamente y un depósito.
- Requisitos: implementar una solución con 1 vehículo (visita a todos los clientes) y con 3 vehículos (balance de rutas), generar demandas aleatorias en Excel entre 50 y 120, y calcular el número de vehículos necesarios si la capacidad por vehículo es 1400 unidades.

3 Generación de datos

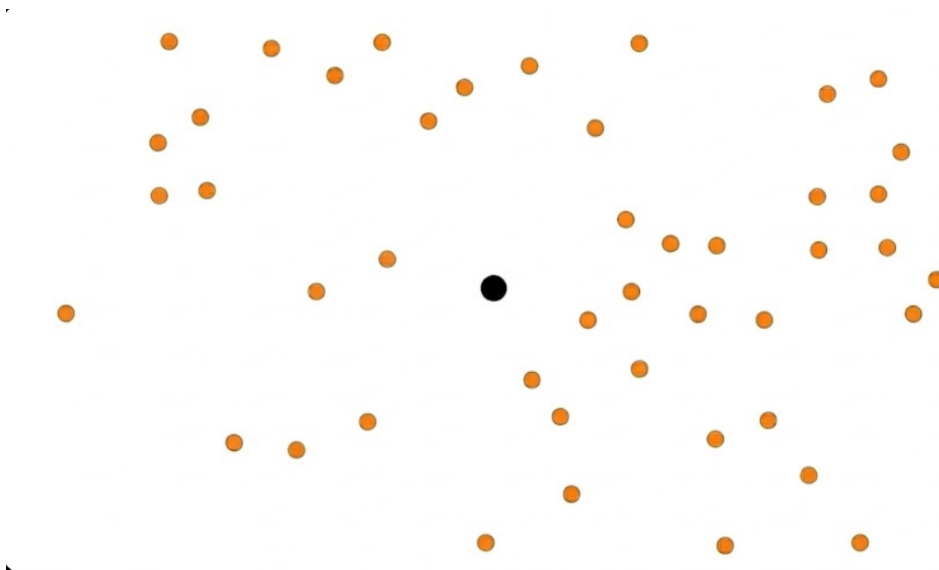


Figure 1: Captura de la plataforma VRP con la nube de 45 clientes y el depósito.

4 Diseño de la ruta con un vehículo

Plantear una única ruta que salga del depósito, visite los 45 clientes y regrese al depósito. Registrar la longitud total de la ruta y el tiempo estimado.

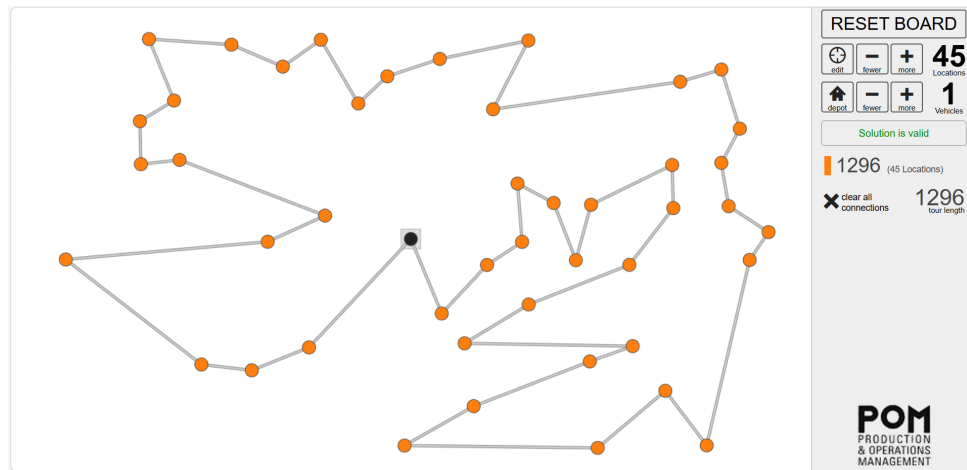


Figure 2: Mapa de la ruta para un vehículo

5 Diseño con flota de tres vehículos

Configurar la instancia con 3 vehículos de capacidad ilimitada (o sin restricción) y obtener una solución balanceada en longitud entre las tres rutas. Documentar la longitud de cada ruta y el criterio usado para balancear (minimizar la máxima longitud, suma total, etc.).

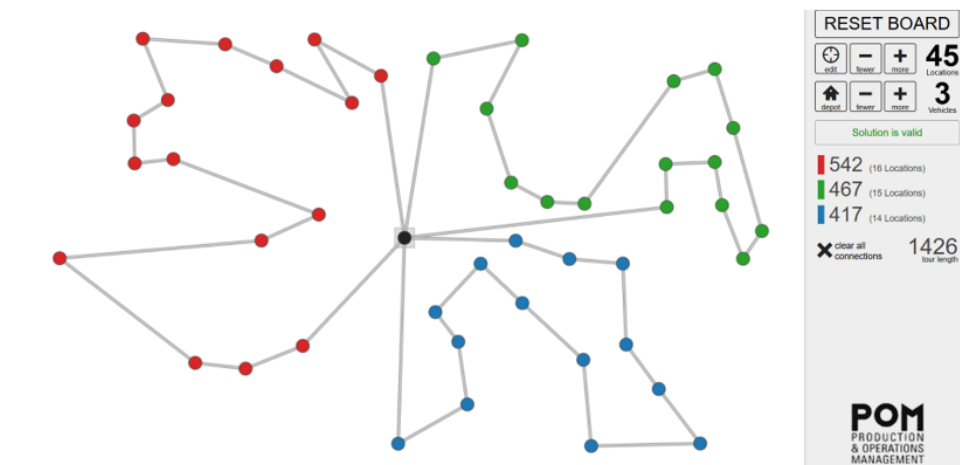


Figure 3: Distribución de las 3 rutas sobre el mapa. En la imagen se indica cómo se repartieron los clientes y se comparan las longitudes de cada ruta para evaluar el balance.

6 Demanda en Excel y cálculo de vehículos necesarios

En Excel se generó una demanda aleatoria uniforme entre 50 y 120 para cada cliente. A partir de la suma total de demanda se calcula el número mínimo de vehículos necesarios cuando cada vehículo tiene capacidad de 1400 unidades

	A	B	C	D	E	F	G	H	I	J
1	Cliente	Demanda	Cliente	Demanda						
2	1	87	25	68						
3	2	107	26	89						
4	3	72	27	109		Total demanda	4080			
5	4	85	28	66						
6	5	78	29	93		Vehiculos necesarios	2,91428571	≈	3	
7	6	94	30	90						
8	7	70	31	94						
9	8	90	32	108						
10	9	106	33	111						
11	10	95	34	70						
12	11	110	35	105						
13	12	100	36	113						
14	13	103	37	117						
15	14	96	38	80						
16	15	95	39	96						
17	16	100	40	118						
18	17	50	41	71						
19	18	97	42	79						
20	19	96	43	110						
21	20	80	44	75						
22	21	103	45	80						
23	22	90								
24	23	84								
25	24	50								

Figure 4: Captura de la hoja Excel con las demandas generadas aleatoriamente y la suma total.

7 Mapeo a ciudad real y ajuste de rutas

7.1 Primer Código: Extracción de Coordenadas (OpenCV)

Para esta sección se realizó un script que actúa como un “escáner” que busca colores específicos en la imagen y registra su ubicación exacta.

- **Cambio de espacio de color (HSV):** Convierte la imagen de BGR (estándar de cámaras) a HSV (*Hue, Saturation, Value*). Esto es crucial porque en HSV es mucho más fácil aislar un color (el “matiz”) sin que la iluminación afecte el resultado.
- **Creación de Máscaras:** Define rangos para el Rojo, Verde y Azul (los nodos generados). La función `cv2.inRange` genera una imagen binaria donde los píxeles blancos representan el color buscado.
- **Detección de Centros:**
 - Utiliza `cv2.findContours` para encontrar las siluetas de los puntos.
 - Calcula el centro de cada punto mediante `cv2.minEnclosingCircle`.
 - **Filtro de ruido:** Si el radio del objeto es menor a 4 píxeles, se descarta para evitar falsos positivos por ruido o texto.
- **Resultado:** Genera tres listas de coordenadas (x, y) correspondientes a cada color detectado.

7.2 Segundo Código: Limpieza y Agrupamiento (Pandas & Scikit-learn)

Este script procesa las coordenadas obtenidas anteriormente y las refina mediante algoritmos estadísticos.

- **Estructuración de datos:** Convierte las listas de coordenadas en un *DataFrame* de Pandas para facilitar su manipulación matricial.
- **Filtro Espacial:** `df = df[df["x"] < 1500]` Esta instrucción elimina cualquier punto ubicado a la derecha del píxel 1500, permitiendo **ignorar el panel de control** lateral y centrarse exclusivamente en el área del mapa.
- **Algoritmo DBSCAN (Clustering):** Corrige errores de detección donde un solo punto físico es interpretado como múltiples puntos cercanos.
 - **eps=25:** Establece que si dos puntos están a una distancia menor a 25 píxeles, pertenecen al mismo objeto.
 - **Promedio por Cluster:** Tras agruparlos, el código calcula el promedio (*mean*) de las coordenadas del grupo, obteniendo un único centroide preciso por cada círculo.
- **Resultado Final:** Una base de datos optimizada de puntos únicos, clasificados por color y con coordenadas espaciales exactas.

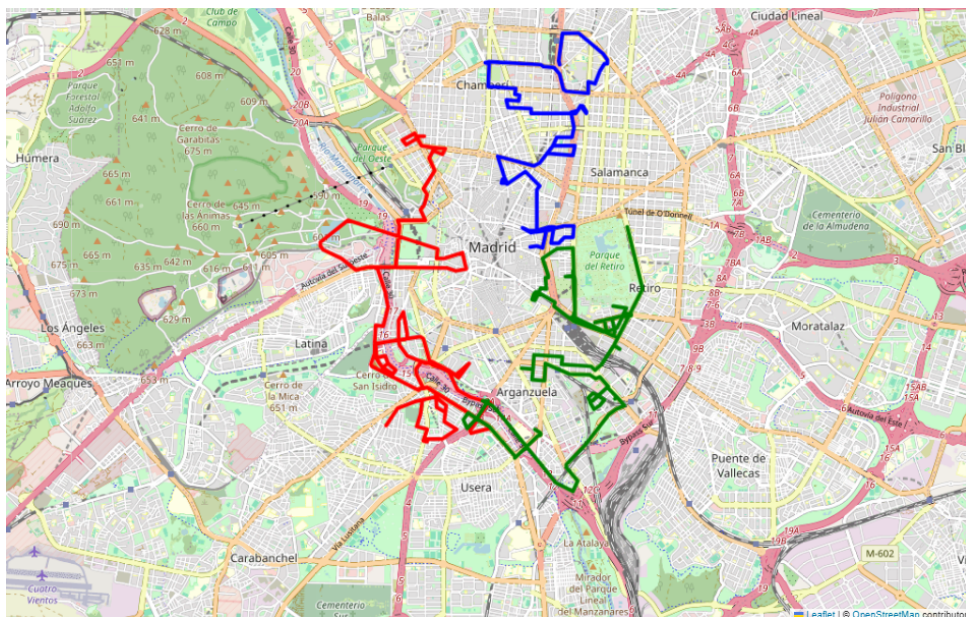


Figure 5: Mapa con la nube de clientes mapeada a la ciudad de Madrid y las rutas sobre la red vial real.

ID	x	y	Color	x_norm	y_norm
0	476.0	710.0	Rojo	0.264558	0.810409
1	377.0	698.0	Rojo	0.193386	0.795539
2	590.0	664.0	Rojo	0.346513	0.753408
3	108.0	491.0	Rojo	0.000000	0.539033
4	508.0	456.0	Rojo	0.287563	0.495663
5	621.0	405.0	Rojo	0.368799	0.432466
6	257.0	303.0	Rojo	0.107117	0.306072
7	333.0	295.0	Rojo	0.161754	0.296159
8	255.0	218.0	Rojo	0.105679	0.200743
9	687.0	183.0	Rojo	0.416247	0.157373
10	323.0	178.0	Rojo	0.154565	0.151177
11	745.0	130.0	Rojo	0.457944	0.091698
12	538.0	111.0	Rojo	0.309130	0.068154
13	436.0	67.0	Rojo	0.235802	0.013631
14	613.0	58.0	Rojo	0.363048	0.002478
15	273.0	56.0	Rojo	0.118620	0.000000
16	1461.0	492.0	Verde	0.972682	0.540273
17	1499.0	437.0	Verde	1.000000	0.472119
18	1310.0	389.0	Verde	0.864127	0.412639
19	1420.0	386.0	Verde	0.943206	0.408922
20	1147.0	383.0	Verde	0.746945	0.405204
21	1074.0	379.0	Verde	0.694464	0.400248
22	1002.0	341.0	Verde	0.642703	0.353160
23	1308.0	304.0	Verde	0.862689	0.307311
24	1405.0	300.0	Verde	0.932423	0.302354
25	1442.0	233.0	Verde	0.959022	0.219331
26	954.0	195.0	Verde	0.608196	0.172243
27	1324.0	140.0	Verde	0.874191	0.104089
28	1405.0	116.0	Verde	0.932423	0.074349
29	848.0	95.0	Verde	0.531991	0.048327
30	1024.0	59.0	Verde	0.658519	0.003717
31	1161.0	863.0	Azul	0.757009	1.000000
32	1376.0	858.0	Azul	0.911574	0.993804
33	778.0	858.0	Azul	0.481668	0.993804
34	915.0	780.0	Azul	0.580158	0.897150
35	1294.0	750.0	Azul	0.852624	0.859975
36	1145.0	692.0	Azul	0.745507	0.788104
37	1230.0	662.0	Azul	0.806614	0.750929
38	897.0	656.0	Azul	0.567218	0.743494
39	852.0	598.0	Azul	0.534867	0.671623
40	1024.0	580.0	Azul	0.658519	0.649318
41	1223.0	502.0	Azul	0.801582	0.552664
42	942.0	502.0	Azul	0.599569	0.552664
43	1118.0	493.0	Azul	0.726096	0.541512
44	1011.0	456.0	Azul	0.649173	0.495663

Table 1: Dataset de puntos extraídos con normalización por eje.

8 Selección y análisis de rutas

Seleccionar revisar detalladamente su recorrido: lista de clientes visitados, distancia por tramo y tiempo.

8.1 Preparación del Entorno Geoespacial

El sistema utiliza OSMnx para descargar el grafo de calles de **Madrid, España**. Se aplica un filtro de tipo `drive` para asegurar que el grafo solo contenga vías aptas para vehículos. Además, se enriquecen los arcos del grafo con atributos de velocidad y tiempo de viaje:

$$T_{travel} = \frac{L_{edge}}{V_{max}} \quad (1)$$

Donde L es la longitud del segmento y V la velocidad permitida.

8.2 Normalización y Mapeo (Píxeles a Coordenadas GPS)

Dado que los datos de entrada provienen de un espacio bidimensional de píxeles, se aplica una transformación lineal para proyectarlos sobre coordenadas geográficas reales (Latitud/Longitud):

- **Normalización:** Se escalan las coordenadas originales al rango $[0, 1]$.

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}, \quad y_{norm} = \frac{y - y_{min}}{y_{max} - y_{min}} \quad (2)$$

- **Georreferenciación:** Se aplica un desplazamiento respecto a un centroide (ϕ_c, λ_c) de Madrid y un factor de escala σ :

$$Lat = \phi_c + (y_{norm} - 0.5) \cdot \sigma, \quad Lon = \lambda_c + (x_{norm} - 0.5) \cdot \sigma \quad (3)$$

8.3 Resolución de Rutas y Métricas Logísticas

Para cada vehículo (identificado por color), el script realiza un mapeo al nodo más cercano en el grafo real mediante `nearest_nodes`. Posteriormente, se calcula el camino mínimo utilizando el algoritmo de Dijkstra:

- **Distancia Real:** Sumatoria de las longitudes de los arcos en metros.
- **Tiempo Estimado:** Basado en el flujo de la red vial.

8.4 Visualización Cartográfica

Finalmente, se genera un mapa interactivo con Folium que incluye:

1. **Depósito Central:** Marcador tipo `home` en el origen de las operaciones.
2. **Clientes:** Representados mediante `CircleMarker` diferenciados por color.

Componente	Función Principal
OSMnx	Modelado y descarga de la red vial de OpenStreetMap.
NetworkX	Cálculo de rutas mínimas y análisis de grafos.
Folium	Renderizado de mapas interactivos y capas visuales.
Pandas	Gestión y transformación de datos tabulares.

Table 2: Stack tecnológico utilizado en la fase de geolocalización.

3. **Rutas:** Representadas por PolyLine que siguen la geometría real de las calles del grafo.

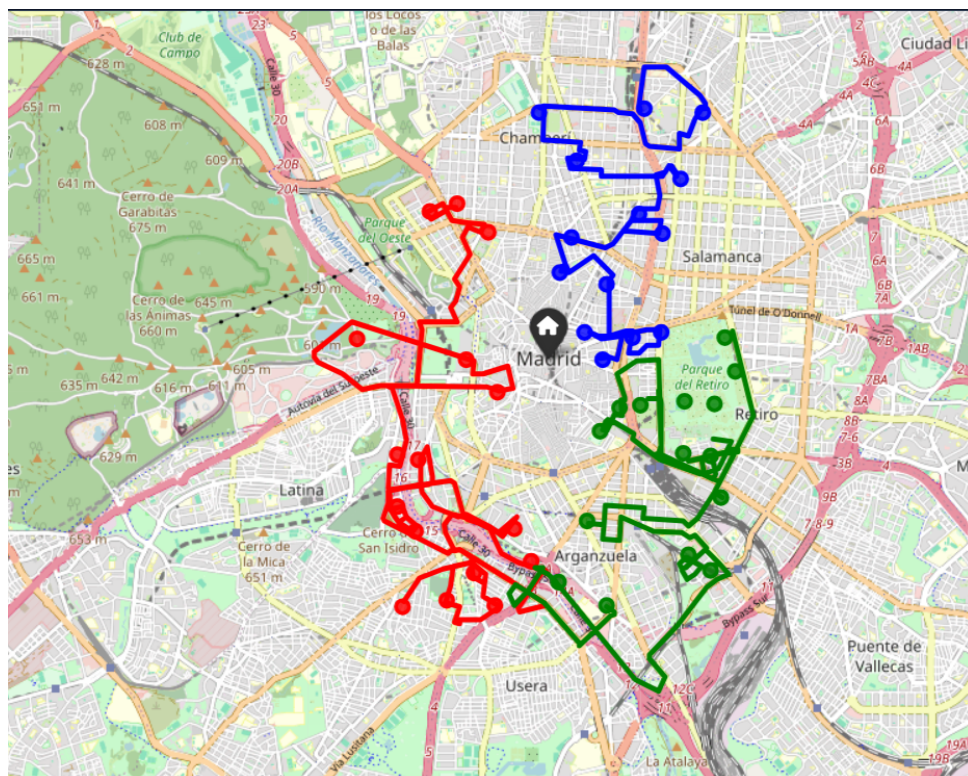


Figure 6: Análisis detallado de dos rutas seleccionadas: mostrar tramos, distancias por tramo y puntos críticos donde se sugiere reorganizar visitas.

9 Entregables

- **Código Fuente:** Scripts completos en github debidamente documentados, incluyendo las fases de procesamiento de imágenes, limpieza de datos y ruteo geoespacial.

Enlace de acceso al repositorio (Código Fuente):
<https://github.com/Gdearmascasta/.../TallerDYT.ipynb>

- **Aplicación Ejecutable:** Se ha desplegado una herramienta interactiva (dashboard) accesible vía web para facilitar la visualización de los resultados. Esta interfaz permite al usuario interactuar directamente con el mapa dinámico.

Enlace de acceso al ejecutable:

<https://rutadyt.netlify.app/>

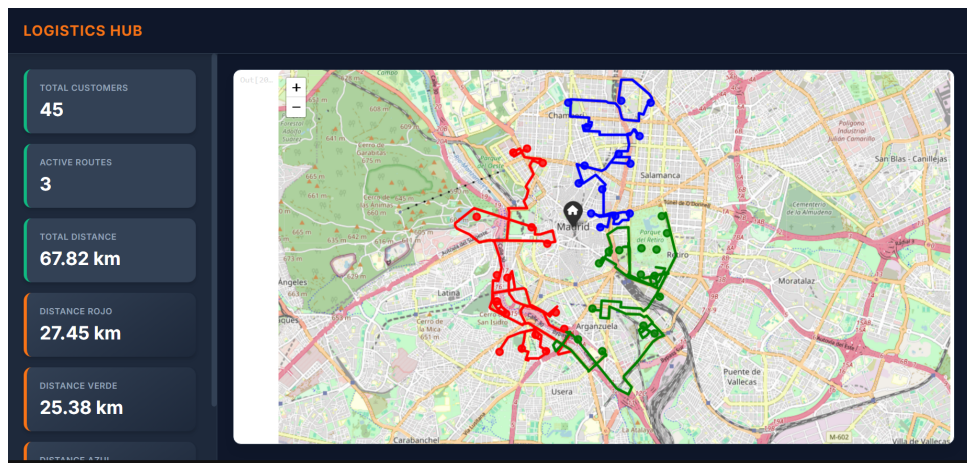


Figure 7: Interfaz del Dashboard Interactivo para la Gestión de Rutas.

9.1 Selección e Identificación de Recorridos

Basado en los resultados obtenidos en el dashboard interactivo (Ver Figura 7), se procedió a la revisión de dos rutas específicas para validar la coherencia del algoritmo de ruteo:

- **Ruta Roja (Sector Sur-Centro):** Con una longitud de **27.45 km**, esta ruta gestiona la mayor carga de transporte. El recorrido identifica paradas estratégicas en distritos del sur de Madrid, optimizando el tránsito a través de vías colectoras y calles secundarias para cubrir la densidad de clientes en Latina y Usera.
- **Ruta Azul (Sector Norte):** Comprende un recorrido de **14.99 km**. Se identifica una trayectoria orientada al norte de la ciudad, cubriendo puntos de entrega en la zona de Chamartín. El diseño de esta ruta aprovecha la estructura radial de la ciudad para minimizar los tiempos de viaje entre nodos.

Se optó por la priorización de la Ruta Azul por encima de la Ruta Verde debido a su superior indicador de eficiencia logística. Aunque la diferencia en el volumen de entregas es mínima (un solo cliente), el ahorro geoespacial es de 10 km. Al comparar ambas alternativas, la Ruta Azul ofrece un mejor ratio de kilómetros por punto de entrega, optimizando el tiempo de tránsito global del sistema.